

Valokaari

Paperinen työpaketti

Päivänvalon visualisointi Suomen paikkakunnille: PHP-rajapinta, React-käyttöliittymä ja julkaisu tuotantoon

18 työviikkoa · päivätön aikataulu · luovutus 18. työviikon perjantai

Tämä paketti on aikataulu ja tarkistuslista tilanteisiin, joissa sivusto ei ole auki. Projektipäiväkirja kirjoitetaan sivustolla ja viedään repositoryn project-docs-kansioon. Rasti tässä vihossa ei ole palautus: työ on aina Git-repositoryssa.

Aikataulu on päivätön: työviikko 1 on se viikko, jolla opiskelija aloittaa, ja projekti kestää 18 työviikkoa.

Julkiseen repositoryyn ei laiteta henkilötietoja, koulun tunnisteita eikä muiden nimiä. Tekijänimestä sovitaan ohjaajan kanssa.

Aikataulu

Vko	Ajoitus	Viikon aihe	Vaihe
1	Työviikko 1 / 18	Aloitius	A
2	Työviikko 2 / 18	Suunnitelma ja priorisointi	A
3	Työviikko 3 / 18	Julkaistu runko	A
4	Työviikko 4 / 18	Päivänvalolaskenta	A
5	Työviikko 5 / 18	Paikkakuntahaku ja rajapinta	A
6	Työviikko 6 / 18	Ensimmäinen kaavio	B
7	Työviikko 7 / 18	Monta paikkaa: itse rakennettu näkymä	B
8	Työviikko 8 / 18	Tooltip ja ääripää	B
9	Työviikko 9 / 18	Leikkauspisteet	B
10	Työviikko 10 / 18	Asiakaskatselmointi	B
11	Työviikko 11 / 18	Vuosivalinta ja palautemuutos kahdessa haarassa	C
12	Työviikko 12 / 18	Mobiili ja saavutettavuus	C
13	Työviikko 13 / 18	Tietoturva-arvio ja virheen käsittely	C
14	Työviikko 14 / 18	Testaus	C
15	Työviikko 15 / 18	Laatu ja dokumentaatio	C
16	Työviikko 16 / 18	Julkaisuehdokas ja julkaisutestaus	D
17	Työviikko 17 / 18	Julkaisu v1.0 ja luovutus	D
18	Työviikko 18 / 18	Näyttö	D

Palautus viimeistään 18. työviikon perjantai. Sivusto: tehtävien tarkat ohjeet, toteutusavut ja projektipäiväkirja.

Vaihe A – Ydin: työkalut, suunnitelma, julkaistu runko ja oikea päivänvalodata

Työviikko 1 / 18 – Aloitus

PHP:n sisäänrakennettu palvelin ja React + Vite -runko käynnistyvät omalla koneella, ja julkinen repository on olemassa ja tarkistettu.

- ☐ Asenna työkalut (PHP 8.2 tai uudempi, Node LTS, Git, VS Code) ja kirjaa versiot README:hen.
- ☐ Luo GitHub-repository ja tee julkisen repon tarkistukset: yksityisyys, tekijänimi ja alaikäisen huoltajan suostumus ohjaajan kautta.
- ☐ Luo React + TypeScript -projekti Vitellä ja käynnistä kehityspalvelin. Käynnistä PHP:n sisäänrakennettu palvelin, joka palauttaa {"status":"ok"}.
- ☐ Lue toimeksianto, kirjoita kysymyslista ohjaajalle (vähintään kuusi kysymystä) ja luo kansiorakenne. Sitten ensimmäinen commit ja push.

Valmis kun: Nimetty toinen henkilö (luokkakaveri tai ohjaaja) saa rungon käyntiin pelkän README:n avulla; ohjaaja on kuitannut julkisen repon tarkistuslistan ja hänet on lisätty repositoryn yhteistyökumppaniksi; kysymyslistassa on vähintään kuusi kysymystä.

Työnäyte Git-repositoryyn ennen rastia: Näytä: README:n versiotaulukko, kuvakaappaukset kehityspalvelimesta ja PHP-vastauksesta, kuitattu julkisuustarkistuslista, kysymyslista ja commit-historian alku (p1, k1; s12 alkaa).

Työviikko 2 / 18 – Suunnitelma ja priorisointi

Hyväksytty tekninen suunnitelma: ohjaajan vastaukset kysymyslistaan, priorisoidut käyttäjätarinat, kaksi vertailua omalla koeajolla, rautalangat, komponenttijako ja issue-tili työmääräarvioineen.

- ☐ Kirjaa ohjaajan vastaukset kysymyslistaan (nimi, päivä, vastaus omin sanoin) ja kirjoita käyttäjätarinat hyväksymiskriteereineen. Priorisoi ne P0/P1/P2 ohjaajan kanssa.
- ☐ Vertaa laskentatapoja omalla koeajolla (date_sun_info Helsingille 21.6.) ja paikkakuntatiedon muotoja (JSON-tiedosto vai SQLite). Päätä ohjaajan kanssa ja perustele.
- ☐ Piirrä rautalangat lomakkeesta, paikkalistasta ja kaaviosta sekä komponenttijako.
- ☐ Pilko P0 issueiksi työmääräarvioineen ja täytä suunnitelma.

Valmis kun: project-docs/suunnitelma.md on repossa ja ohjaaja on hyväksynyt rajauksen kirjatulla kommentilla; ohjaajan vastaukset kysymyksiin on kirjattu nimellä ja päivällä; jokaisella P0-tarinalla on GitHub-issue, hyväksymiskriteerit ja arvio tunteina; koeajon tuloste on päiväkirjassa.

Työnäyte Git-repositoryyn ennen rastia: Näytä: project-docs/suunnitelma.md, ohjaajan vastaukset, koeajon tuloste päiväkirjassa, kaksi vertailutaulukkoa, rautalangat, issue-tili ja ohjaajan kirjattu hyväksyntä (s1, s4, s5, s8, p8, p9; k5 ja s6 alkavat).

Työviikko 3 / 18 – Julkaistu runko

Sovellus, jossa on yksi oma React-komponentti, ja PHP-rajapinnan /api/health ovat julkisessa osoitteessa; kehityksessä Viten proxy ohjaa /api-kutsut PHP:hen; rajapintasopimus ja React-selvitys ovat repossa.

- Toteuta GET /api/health PHP:llä ja ohjaa Viten kehityspalvelimen /api-kutsut PHP:hen. Kirjoita api-sopimus.md: osoite, parametrit, vastauksen JSON-muoto ja virhevastaukset.
- Kirjoita ensimmäinen oma komponentti HealthStatus, joka hakee /api/health-vastauksen ja näyttää tilan.
- Valitse julkaisumalli suunnitelman kriteereillä ja julkaise sovellus ja rajapinta ohjaajan antamaan alustaan. Jos julkaisu ei onnistu torstaihin mennessä, kirjaa virhe ja alustan loki päiväkirjaan ja jatka viikkoon 4.
- Kirjoita selvitys taulukkona omista P0-tarinoista: mikä osa tehdään Reactilla ja Vitellä, mikä tulee ulkopuolelta ja mistä. Pyydä toista henkilöä avaamaan julkinen osoite omalla laitteellaan.

Valmis kun: Toinen henkilö avaa julkisen osoitteen ja /api/health omalla laitteellaan ja näkee HealthStatus-komponentin ilmoittavan "ok"; alustan lokinäkymästä on kuvakaappaus; build tehdään alustalla ja client/dist on gitignoresa; api-sopimus.md ja selvitystaulukko ovat repossa.

Työnäyte Git-repositoryyn ennen rastia: Näytä: julkinen osoite, kuvakaappaus alustan lokinäkymästä, vite.config.ts:n proxy-asetus, HealthStatus.tsx, project-docs/api-sopimus.md ja react-vite-selvitys.md, julkaisuperustelu ja toisen henkilön kiittäminen (k2; k1 täydentyi; k3, s10, s14 ja k6 alkavat).

Työviikko 4 / 18 – Päivänvalolaskenta

PHP-moduuli Daylight laskee yhden päivän päivänvalon minuutit koordinaateille ja koko vuoden listan; arvot pysyvät välillä 0–1440, karkausvuosi tuottaa 366 päivää, ja testit T01–T04 menevät läpi komennolla.

- Hae odotusarvot ulkoisesta lähteestä ja kirjaa testitapaukset T01–T04 ennen koodia.
- Toteuta Daylight-moduuli: yhden päivän minuutit koordinaateille, koko vuoden lista, rajat 0–1440 ja karkausvuosi.
- Aja testit komennolla, vertaa PHP:n date_sun_info-referenssiin ja korjaa poikkeamat. Kirjaa jokainen löytynyt virhe heti bug-issueksi.

Valmis kun: Testit T01–T04 menevät läpi komennolla (`composer test` tai `php tests/run.php`); vuosi 2028 palauttaa 366 riviä ja 2027 365; mikään arvo ei ole alle 0 tai yli 1440; poikkeamat referenssiin on kirjattu ja selitetty; löydetty virheet ovat bug-issueina.

Työnäyte Git-repositoryyn ennen rastia: Näytä: server/src/Daylight.php, testitiedosto ja testiajon tuloste (T01–T04 odotusarvoineen ja lähteineen), vertailu referenssiin, bug-issues ja commitit (p4, s7; p3 alkaa; p2:n raaka-aine).

Työviikko 5 / 18 – Paikkakuntahaku ja rajapinta

GET /api/daylight?city=&year= palauttaa sopimuksen mukaisen JSONin oikealle paikkakunnalle; tuntematon nimi antaa 404-virheen ja virheellinen syöte 400-virheen JSONina.

- Ota paikkakuntadata käyttöön valitsemassasi muodossa (JSON tai SQLite) ja kirjaa tietolähteen lisenssi. Jos oma hankinta ei valmistu tiistaihin mennessä, käytä ohjaajan kuntaluetteloa.
- Toteuta haku nimellä: isot ja pienet kirjaimet, ä ja ö sekä ylimääräiset välilyönnit.
- Toteuta GET /api/daylight?city=&year= sopimuksen mukaan. Validoi syötteet ja palauta virheet 400 ja 404 JSONina.
- Aja T05–T07 curl-komennolla ja kirjaa odotettu ja saatu vastaus.

Valmis kun: T05 (Helsinki 2026 → 200, 365 riviä, nimi ja koordinaatit), T06 ("utsjoki" pienillä → 200 Utsjoki) ja T07 ("Tuntematonkylä" → 404 JSON-virhe) antavat curlilla kirjatut vastaukset; paikkakuntatiedoston lähde ja lisenssi ovat README:ssä; syötteiden pituus ja vuosiväli tarkistetaan.

Työnäyte Git-repositoryyn ennen rastia: Näytä: server/src/Geocoder.php ja rajapinnan reititys, paikkakuntatiedosto lisenssimerkintöineen, curl-tulosteet T05–T07 ja commitit (s9; s7 täydentyi; p7 alkaa; s10, jos käytät ulkoista rajapintaa).

Vaihe B – Ominaisuudet: kaavio, monta paikkaa, tooltip, leikkauspisteet ja asiakaskatselmointi

Työviikko 6 / 18 – Ensimmäinen kaavio

Selain hakee Helsingin datan omasta rajapinnasta ja piirtää sen viivakaaviona; lataus- ja virhetila näkyvät; kaaviokirjasto on valittu vertailulla, ja työ tehtiin ensimmäisessä ominaisuushaarassa.

- Vertaa kolmea kaaviokirjastoa (koko omasta npm-tulosteesta, lisenssi, React-tuki, tooltip- ja merkintätuki, näppäimistötuki), perustele valinta ja kirjaa se riippuvuustaulukkaan.
- Luo ominaisuushaara, toteuta api.ts (rajapintakutsu fetchillä, vastauksen tyyppi, virheen käsittely) erillään kaaviosta ja piirrä Helsingin vuosi viivakaaviona lataus- ja virhetiloineen.
- Avaa pull request, katselmoi se itse, yhdistä ja julkaise. Kirjaa ja aja T16: sammuta palvelin ja tarkista, että selain näyttää virheilmoituksen eikä tyhjää sivua.

Valmis kun: Kaavio näkyy julkaistussa versiossa; api.ts sisältää kaikki rajapintakutsut ja vastauksen TypeScript-tyypin; T16 on kirjattu ja ajettu: kun PHP-palvelin sammutetaan, selain näyttää virheilmoituksen eikä tyhjää sivua; ensimmäinen pull request on yhdistetty itsekselmoittimen jälkeen.

Työnäyte Git-repositoryyn ennen rastia: Näytä: client/src/api.ts, DaylightChart-komponentti, kaaviokirjastoverailu kokeilusarakeineen, ensimmäinen pull request, kuvakaappaus julkaistusta kaaviosta ja T16:n virhetilasta (k4, s10, k3, s12; p6 alkaa).

Työviikko 7 / 18 – Monta paikkaa: itse rakennettu näkymä

Lomake lisää paikan, lista näyttää mukana olevat paikat poistonapilla, sama paikka ei tule kahdesti, jokaisella sarjalla on oma väri ja kaikki paikat näkyvät samalla 0–24 h asteikolla. Lomake ja lista on rakennettu alusta itse Tailwindilla.

- Ota Tailwind käyttöön perusteltuna ulkoisena komponenttina ja rakenna lomake ja paikkalista alusta ilman valmista komponenttikittiä.
- Estä sama paikka kahdesti, anna jokaiselle sarjalle oma väri ja pidä asteikko 0–24 h kaikilla paikoilla.
- Kirjaa ja aja T08 (neljä paikkaa samassa kaaviossa) ja T09 (sama paikka toistamiseen hylätään ilmoituksella).
- Vertaa viikkopalaverien työmääräarvioita toteumaan ja päivitä suunnitelma, jos arvio petti.

Valmis kun: T08: Helsinki, Tampere, Rovaniemi ja Utsjoki näkyvät samassa kaaviossa eri väreillä ja yhteisellä asteikolla; T09: sama paikka toistamiseen hylätään ilmoituksella; paikan voi poistaa listasta; Tailwind on riippuvuustaulukossa perusteltuna; arvio vs. toteuma -taulukko on päiväkirjassa.

Työnäyte Git-repositoryyn ennen rastia: Näytä: useLocations-hook, LocationForm- ja LocationList-komponentit, Tailwind-perustelu riippuvuustaulukossa, T08–T09:n tulokset kuvakaappauksineen, arvio vs. toteuma -taulukko (p6, p7, k3, k4, s6).

Työviikko 8 / 18 – Tooltip ja ääripäät

Tooltip näyttää päivän, paikan ja keston luettavassa muodossa, ja kaamos (0 min) sekä yötön yö (1440 min) erottuvat kaaviosta.

- Toteuta format.ts (minuutit → 23 h 59 min, päivä → 21. kesäkuuta) ja sen yksikkötestit T10–T11 ennen kaavioon kytkemistä.
- Toteuta tooltip: päivä, paikka ja kesto luettavassa muodossa hiirellä ja kosketuksella. Kirjaa, mitä kirjasto tukee näppäimistöllä, ja suunnittele kiertoreitti viikolle 12.
- Korosta kaamos (0 min) ja yötön yö (1440 min) kaaviossa Utsjoen ja Rovaniemen datalla.

Valmis kun: T10 ja T11 menevät läpi komennolla `npm test`; tooltip toimii hiirellä ja kosketuksella ja näyttää esimerkiksi "21. kesäkuuta · Rovaniemi · 24 h"; Utsjoen kaamosjaksot ja yöttömän yön jaksot erottuvat kaaviosta; päiväkirjassa on kirjattu, mitä kirjasto tukee näppäimistöllä, ja päätös kiertoreitistä (datataulukko vai päivävalitsin) perusteluineen.

Työnäyte Git-repositoryyn ennen rastia: Näytä: client/src/format.ts ja sen testit, tooltip-komponentti, kirjattu näppäimistörajoite ja kiertoreitti, kuvakaappaus Utsjoen kaaviosta korostuksineen (p3, k3, k2; p4 täydenty).

Työviikko 9 / 18 – Leikkauspisteet

Sovellus laskee ja merkitsee kaavioon päivät, joina kahden paikan päivänvalo on yhtä pitkä tai lähes yhtä pitkä, ja merkin tooltip nimeää päivän, molemmat paikat ja keston.

- ☐ Kirjoita leikkauspisteen määritelmä ja toleranssi suunnitelmaan ja perustele se Utsjoki–Inari-esimerkillä.
- ☐ Toteuta intersections.ts puhtaana funktiona, joka yhdistää peräkkäiset päivät yhdeksi merkiksi. Kirjaa testit T12–T13 ennen ajoa.
- ☐ Piirrä merkit kaavioon ja anna niille oma tooltip: päivä, molemmat paikat ja kesto.

Valmis kun: T12: Helsinki–Rovaniemi antaa leikkauksen maaliskuu- ja syyskuussa, ei kesäkuussa; T13: Utsjoki–Inari antaa yöttömän yön ajalta yhden merkin, ei kymmeniä; testit menevät läpi komennolla; kaaviossa on merkki, jonka tooltip nimeää molemmat paikat ja päivän; määritelmä ja toleranssi ovat suunnitelmassa perusteltuina.

Työnäyte Git-repositoryyn ennen rastia: Näytä: client/src/intersections.ts ja sen testit, suunnitelman päätöskohta, kuvakaappaus Helsinki–Rovaniemi-leikkauksesta maaliskuussa (s7, p4, p3).

Työviikko 10 / 18 – Asiakaskatselmointi

Nimetty ulkopuolinen henkilö on kokeillut julkaistua väliversiota asiakkaan roolissa, ja hänen havaintonsa ovat muistiossa sitaatteina erillään omasta tulkinnastasi, muutokset issueina.

- ☐ Valmistele asiakaskielinen esittely ilman teknistä sanastoa ja kolmen paikan testitehtävä katselmoijalle. Herätä nukkukuva julkaisualusta ennen tilaisuutta.
- ☐ Pidä katselmointi nimetyn ulkopuolisen kanssa julkaistulla väliversiolla. Kirjaa hänen sanansa sitaatteina.
- ☐ Kirjoita muistio: sitaatit, oma tulkinta erikseen ja priorisoidut muutokset issueiksi. Rajaa ohjaajan kanssa tärkein muutos enintään kahden päivän työksi.

Valmis kun: Katselmointimuistio on repossa: katselmoijan rooli ja ajankohta, vähintään viisi sitaattia, oma tulkinta erillään, priorisoidut muutokset issueina P-luokin; ohjaajan kanssa on rajattu tärkein muutos enintään kahden päivän työksi työviikolle 11.

Työnäyte Git-repositoryyn ennen rastia: Näytä: project-docs/katselmointi-vk10.md (rooli, ajankohta, sitaatit, tulkinta) ja muutosissuet P-luokin (s2, s3, p10; s1 täydentyä).

Vaihe C – Valmiiksi: vuosivalinta ja palautemuutos, mobiili ja saavutettavuus, tietoturva, testaus ja laatu

Työviikko 11 / 18 – Vuosivalinta ja palautemuutos kahdessa haarassa

Käyttäjä valitsee vuoden ja kaikki paikat haetaan uudelleen; katselmoinnin tärkein muutos on toteutettu; molemmat tehtiin omissa haaroissa, ja niiden välinen merge-konflikti on ratkaistu hallitusti.

- ☐ Luo maanantaina kaksi haaraa samasta main-commitista: vuosivalinta ja palautemuutos.
- ☐ Toteuta ensin vuosivalitsin, joka hakee kaikkien paikkojen datan uudelleen, ja aja T14 (vuosi 2028 näyttää 366 pistettä). Yhdistä pull requestilla.
- ☐ Toteuta palautemuutos toisessa haarassa, ratkaise syntyvä merge-konflikti hallitusti, yhdistä ja julkaise. Jos konfliktia ei synny, ohjaaja tekee sovitun commitin main-haaraan.

Valmis kun: Kaksi pull requestia on yhdistetty; konfliktin ratkaisun commit näkyy historiassa; T14: vuosi 2028 näyttää tuotannossa 366 pistettä ja 2027 365; katselmointimuistion päätös on kuitattu linkillä; jos konfliktia ei syntynyt, ohjaajan commit ja sen ratkaisu on kirjattu.

Työnäyte Git-repositoryyn ennen rastia: Näytä: kaksi pull requestia, konfliktin ratkaisun commit, YearSelect-komponentti, T14:n tulos ja muutokset julkaistussa versiossa (s13, s10, p7; s12 koonti; p10 täydentyy).

Työviikko 12 / 18 – Mobiili ja saavutettavuus

Sovellus toimii puhelimella ja näppäimistöllä: asettelu mukautuu, työviikon 8 kiertoreitti on toteutettu, ja Lighthouse-raportit ennen ja jälkeen osoittavat korjaukset.

- ☐ Testaa sovellus oikealla puhelimella ja korjaa asettelu Tailwindin taitekohdilla. Kuvakaappaukset ennen ja jälkeen.
- ☐ Toteuta viikolla 8 valittu näppäimistön kiertoreitti (datataulukko tai päivävalitsin) ja tarkista näppäinkäyttö, kontrasti, label-kytkennät ja otsikkohierarkia.
- ☐ Aja Lighthouse ennen ja jälkeen korjausten ja kirjaa tulokset.

Valmis kun: Sovellus toimii oikealla puhelimella ilman vaakasuuntaista rullausta; jokainen lomakkeen kenttä on saavutettavissa Tab-näppäimellä ja sillä on label; kiertoreitti näyttää valitun päivän arvot kaikille paikoille ilman hiirtä; Lighthouse-saavutettavuuspisteet ennen ja jälkeen on kirjattu ja jälkeen-luku on korkeampi tai syy on selitetty.

Työnäyte Git-repositoryyn ennen rastia: Näytä: kuvakaappaukset omalta puhelimelta, kiertoreitin komponentti, Lighthouse-raportit ennen ja jälkeen (k3, k2; p6 täydentyy).

Työviikko 13 / 18 – Tietoturva-arvio ja virheenkäsittely

Oman sovelluksen uhkalista on kirjoitettu ja testattu: rajapinta hylkää huonot syötteet hallitusti, paikan nimi näytetään turvallisesti, yhden paikan virhe ei kaada muita, eikä repossa ole salaisuuksia.

- ☐ Kirjoita oman sovelluksen uhkalista (5–8 riviä: uhka, testi, tulos, toimenpide) ja aja syötetaulukko T15 curlilla.
- ☐ Tee XSS-testi: lisää paikka nimellä `<script>alert(1)</script>` ja varmista, että nimi näkyy tekstinä eikä suoriteta.
- ☐ Käsittele yhden paikan virhe niin, että muut sarjat säilyvät (Promise.allSettled), ja varmista, ettei repossa ole salaisuuksia. Korjaa vain rivit, joilla saatu poikkeava odotetusta; muut issueiksi.

Valmis kun: Uhkalistassa on 5–8 riviä omasta sovelluksesta muodossa uhka → testi → tulos → toimenpide; T15:n syötetaulukko on ajettu curlilla ja jokainen rivi, jolla saatu poikkeava odotetusta, on korjattu tai issueina; XSS-testi näyttää nimen tekstinä; tuntematon paikka listassa ei kaada muita sarjoja; git log ja tiedostot eivät sisällä avaimia tai salasanoja.

Työnäyte Git-repositoryyn ennen rastia: Näytä: project-docs/tietoturva-arvio.md uhkalistoinen ja syötetaulukkoineen, XSS-testin kuvakaappaus, korjauscommitit (s11, s7).

Työviikko 14 / 18 – Testaus

Testiraportti kokoa 16 testitapausta kolmessa luokassa odotusarvoineen, testit ajetaan komennolla PHP:llä ja TypeScriptillä, ja vähintään kaksi täydellistä virheenkorjausketjua on kirjattu aidosta havainnoista.

- ☐ Kokoa testiraportti: 16 testitapausta kolmessa luokassa, odotusarvot ja lähteet kirjattuina viikoilla 4–13 ennen ajoa; erottele komennolla ajettavat ja käsin ajettavat selaintestit.
- ☐ Aja PHP-testit (composer test tai php tests/run.php) ja Vitest-testit (npm test) ja kirjaa tulokset.
- ☐ Kirjaa vähintään kaksi täydellistä virheenkorjausketjua bug-issuelistan aidosta havainnoista.

Valmis kun: Raportissa on 16 tapausta luokiteltuina (normaali 6, rajat 6, virheet 4) odotusarvoin ja lähtein; komennolla ajettavat (T01–T07, T10–T13, T15) ja käsin ajettavat selaintestit (T08, T09, T14, T16) on eroteltu; `composer test` tai `php tests/run.php` ja `npm test` menevät läpi ja tulosteet ovat repossa; kaksi ketjua on kirjattu aidosta bug-issueista täydellisinä.

Työnäyte Git-repositoryyn ennen rastia: Näytä: project-docs/testiraportti.md, testiajojen tulosteet ja virheenkorjausketjut commit-linkein (p3, p2, k5).

Työviikko 15 / 18 – Laatu ja dokumentaatio

Ohjaajan koodikatselmoinnin kolme havaintoa on refaktoroitu ja dokumentoitu, README kertoo käyttöönoton ja rajapinnan, riippuvuustaulukko on tehty tulosteista ja LICENSE on repossa tai kirjattu avoimeksi.

- ☐ Ota vastaan ohjaajan koodikatselmoinnin kolme nimettyä havaintoa, refaktoroi ne ja dokumentoi ennen ja jälkeen -diffit perusteluineen.
- ☐ Kirjoita README: käyttöönotto tyhjään ympäristöön, kehityskomennot, rajapinnan kuvaus esimerkivastauksella ja riippuvuustaulukko npm ls- ja composer show-tulosteista lisensseineen.
- ☐ Lisää LICENSE ohjaajan päätöksen mukaan. Jos päätöstä ei ole, kirjaa avoin asia.

Valmis kun: Kolme ohjaajan nimeämää havaintoa on refaktoroitu, jokaisesta on diff-linkki, perustelu ja testiajo läpi; README sisältää käyttöönoton tyhjään ympäristöön, kehityskomennot, rajapinnan kuvauksen esimerkivastauksella ja riippuvuustaulukon `npm ls --depth=0`- ja `composer show`-tulosteista lisensseineen; LICENSE on repossa tai suunnitelmassa on avoin asia päivämäärällä.

Työnäyte Git-repositoryyn ennen rastia: Näytä: refaktorointimuistio diff-linkein ja ohjaajan havainnot, README, riippuvuustaulukko ja LICENSE tai kirjattu avoin asia (p5, k7, s12).

Vaihe D – Julkaisu: julkaisuehdokas ja julkaisutestaus, v1.0 ja näyttö

Työviikko 16 / 18 – Julkaisuehdokas ja julkaisutestaus

Sisältö on jäädytetty, julkaisuehdokas v1.0-rc1 on julkaistu ja merkitty Git-tagilla, ja nimetty ulkopuolinen henkilö on testannut sen pelkän kirjallisen ohjeen avulla.

- ☐ Tee jäädytyspäättös ja luokittele jäljellä olevat issueet estäviksi tai v1.1-listalle.
- ☐ Julkaise ja merkitse julkaisuehdokas Git-tagilla v1.0-rc1.
- ☐ Asenna sovellus itse puhtaaseen ympäristöön (toinen kone tai tyhjä kansio) pelkän ohjeen avulla ja korjaa ohje pöytäkirjan perusteella.
- ☐ Anna nimetylle ulkopuoliselle ohje ja osoite. Kirjaa hänen havaintonsa sitaatteina ja estävät virheet issueiksi.

Valmis kun: v1.0-rc1 on julkisessa osoitteessa ja merkitty tagilla; oma asennuspöytäkirja (toinen kone tai tyhjä kansio) ja ulkopuolisen testauspöytäkirja (nimetty rooli, ajankohta, testaajan omat sanat erillään tulkinnasta) ovat repossa; estävät virheet on kirjattu issueiksi, ei korjattu kiireellä tällä viikolla.

Työnäyte Git-repositoryyn ennen rastia: Näytä: tag v1.0-rc1, oma asennuspöytäkirja, julkaisutestauksen pöytäkirja ja estävien issueiden lista (s3, s14, k6).

Työviikko 17 / 18 – Julkaisu v1.0 ja luovutus

Julkaisutestauksen estävät virheet on korjattu täydellisenä virheenkorjausketjuna, v1.0 on julkaistu ja merkitty tagilla, ja sovellus on luovutettu asiakkaan roolissa toimivalle ohjaajalle, joka on vastannut kysymyksellä.

- ☐ Korjaa estävät virheet täydellisenä virheenkorjausketjuna ja aja regressiotestit. Jos estäviä ei löytynyt, ota kolmas ketju bug-listalta.
- ☐ Merkitse v1.0 Git-tagilla, tee release, julkaise ja aja savutesti tuotannossa.
- ☐ Kirjoita julkaisutiedote ja luovutusviesti ohjaajalle asiakkaan roolissa. Vastaa hänen asiakaskysymykseensä.

Valmis kun: v1.0 on julkisessa osoitteessa ja merkitty tagilla; kolmas ketju on täydellisenä repossa (estävästä virheestä tai bug-listan aidosta havainnosta); regressioajo ja savutesti on kirjattu; julkaisutiedote ja luovutusviesti ovat repossa; ohjaajan asiakaskysymys ja vastauksesi on kirjattu; viikolle jäi puskuriaikaa eikä mitään uutta aloitettu.

Työnäyte Git-repositoryyn ennen rastia: Näytä: tag v1.0 ja release, kolmas virheenkorjausketju, savutestin kirjaus, julkaisutiedote.md, luovutusviesti.md ja asiakkaan kysymys vastauksineen (s14, k6, s2, p2).

Työviikko 18 / 18 – Näyttö

Mitään uutta ei rakenneta: näyttöaineisto on täsmälinkitetty kaikkiin 32 vaatimukseen, 8–10 minuutin demo on harjoiteltu toiselle henkilölle ja itsearviointi on kirjoitettu kolmesta nimetystä tilanteesta.

- ☐ Täsmälinkitä näyttömatriisin 32 vaatimusta työnäytteisiin ja korjaa aukot.
- ☐ Harjoittele 8–10 minuutin demo kellon kanssa toiselle henkilölle.
- ☐ Kirjoita itsearviointi kolmesta nimetystä tilanteesta, pyydä ohjaajan vastakommentti ja luovuta näyttöpaketti.

Valmis kun: Jokainen matriisin 32 riviä osoittaa olemassa olevaan aineistoon ja linkki aukeaa; demo on ajettu kellon kanssa vähintään kerran toiselle henkilölle; itsearviointi käsittelee työviikon 10 palautteen, työviikon 16 testaajan epäröinnin ja yhden viikkopalaverin, jossa sovittu muuttui, ja ohjaaja on kirjannut lyhyen vastakommentin.

Työnäyte Git-repositoryyn ennen rastia: Näytä: täsmälinkitetty näyttömatriisi, demorunko, itsearviointi.md ohjaajan kommentilla (p11).

Viimeiset viisi päivää

Ma · Sisältöjäädytys: viimeinen hyväksytty versio, matriisin täsmälinkitys alkaa

Ti · Aineisto: päiväkirja, AI-loki, testiraportti ja linkkien tarkistus

Ke · Demoharjoitus kellon kanssa (8–10 min) toiselle henkilölle

To · Itsearviointi kolmesta tilanteesta ja ohjaajan vastakommentti

Pe · Luovutus: näyttömatriisi, projektipäiväkirja, AI-loki ja julkaistu v1.0

Sanasto

Projektin tunnukset ja ammattitermit siinä järjestyksessä, jossa ne tulevat vastaan. Jokainen on selitetty myös sivustolla ensimmäisen käytön kohdalla.

repository vko 1	<i>Git-repository, repo.</i> Projektin versionhallittu kansio GitHubissa. Kaikki työnäytteet ovat repositoryssa, eivät tällä sivulla.
commit vko 1	<i>Versionhallintaan tallennettu muutos.</i> Yksi nimetty muutoskokonaisuus, jolla on viesti ja tunnus. Pieni commit on helpompi lukea ja perua kuin suuri.
push vko 1	<i>Muutosten vienti etärepositoryyn.</i> Komento, joka siirtää omat commitit GitHubiin. Vasta pushin jälkeen työ on työnäyte.
backend vko 1	<i>Palvelinpuoli.</i> Se osa sovellusta, joka ajetaan palvelimella. Valokaassa PHP-koodi, joka laskee päivänvalon ja palauttaa sen rajapinnan kautta.
frontend vko 1	<i>Selainpuoli.</i> Se osa sovellusta, joka ajetaan käyttäjän selaimessa. Valokaassa React-sovellus, joka piirtää kaavion.
API vko 1	<i>Rajapinta (application programming interface).</i> Sovittu tapa, jolla selain pyytää tietoa palvelimelta. Valokaaren rajapinta on yksi osoite: GET /api/daylight?city=Helsinki&year=2026.
JSON vko 1	<i>Tekstipohjainen tietomuoto.</i> Muoto, jossa rajapinta palauttaa tiedon: avaimia ja arvoja aaltosulkeissa. Sekä PHP että selain osaavat lukea ja kirjoittaa sitä.
kehityspalvelin vko 1	<i>Viten kehityspalvelin.</i> Paikallinen palvelin, joka näyttää React-sovelluksen selaimessa ja päivittää sen heti, kun tallennat tiedoston. Käynnistyy komennolla npm run dev.
GitHub-issue vko 2	<i>Tehtäväkortti GitHubissa.</i> Yksi rajattu työtehtävä, jolla on otsikko, kuvaus, hyväksymiskriteerit ja työmääräarvio. Bug-issue on tunnisteella bug merkitty virrehavainto.
käyttäjätarina vko 2	<i>Vaatuskäyttäjän näkökulmasta.</i> Muotoa ”Matkailijana haluan ... jotta ...”. Jokaisella tarinalla on 2–4 hyväksymiskriteeriä, joista tiedät, milloin se on valmis.
P0 vko 2	<i>Pakollinen ydin.</i> Toiminnot, joiden on valmistuttava ennen mitään lisäominaisuuksia. Valokaassa oikea päivänvalodata, usea paikka, virheilmoitus, tooltip ja ääripäiden korostus.
P1 vko 2	<i>Tärkeä jatko.</i> Tehdään, kun P0 toimii ja on testattu. Valokaassa leikkauspisteet, vuosivalinta, mobiili ja saavutettavuus.
P2 vko 2	<i>Valinnainen lisä.</i> Saa jäädä projektin ulkopuolelle, jos aika ei riitä. Poisjätto on rajauspäätös, ei epäonnistuminen.
rautalanka vko 2	<i>Käyttöliittymän luonnos.</i> Yksinkertainen piirros siitä, mitä näkymässä on ja missä. Ei värejä, ei lopullista ulkoasua. Toteutus tarkistetaan sitä vasten.
viikkopalaveri vko 2	<i>Ohjaajan kanssa joka viikko.</i> 10 minuuttia: sovitut tehtävät, arviot ja edellisen viikon toteuma kirjataan issueihin, ohjaajan nimi ja päivä päiväkirjaan. Tämä on tiimissä sopimisen työnäyte.
proxy vko 3	<i>Välityspalvelin kehityksessä.</i> Viten asetus, joka ohjaa selaimen /api-pyyynnöt kehityspalvelimelta PHP-palvelimelle. Selain näkee yhden osoitteen, vaikka palvelimia on kaksi.
build vko 3	<i>Tuotantobuild.</i> Komento npm run build kääntää React-sovelluksen valmiiksi tiedostoiksi client/dist-kansioon. Kansio ei mene Gitiin: build tehdään julkaisualustalla.
fetch vko 3	<i>Selaimen tapa hakea tietoa.</i> JavaScriptin funktio, joka lähettää pyynnön rajapintaan ja palauttaa vastauksen. Valokaassa kaikki fetch-kutsut ovat api.ts-tiedostossa.
komponentti vko 3	<i>React-komponentti.</i> Itsenäinen käyttöliittymän pala, joka saa tietoa propseina ja voi pitää omaa tilaa. HealthStatus on ensimmäinen, DaylightChart ja LocationList seuraavat.
T01 vko 4	<i>Testitapauksen tunnus.</i> T tarkoittaa testitapausta ja numero yksilöi sen: T01 on ensimmäinen, T16 viimeinen. Odotettu tulos kirjataan aina ennen ajoa.
yksikkötesti vko 4	<i>Yhden funktion testi.</i> Testi, joka kutsuu yhtä funktiota tunnetulla syötteellä ja vertaa tulosta odotettuun. Ajetaan komennolla ilman selainta. PHP:ssä PHPUnit tai oma testiskripti, TypeScriptissä Vitest.
Composer vko 4	<i>PHP:n paketinhallinta.</i> Työkalu, joka asentaa PHP-kirjastot (esimerkiksi PHPUnitin) ja ajaa skriptit komennolla composer test. Vastaa npm:ää PHP-puolella.

bug-issue vko 4	<i>Virrehavainto issueksi.</i> Jokainen havaittu virhe kirjataan heti GitHub-issueksi tunnisteella bug: mitä odotit, mitä tapahtui. Virheenkorjausketjut otetaan tältä listalta.
geokoodaus vko 5	<i>Nimestä koordinaateiksi.</i> Paikkakunnan nimen muuttaminen leveys- ja pituusasteiksi. Valokaareissa se tehdään omasta paikkakuntatiedostosta; ulkoinen geokoodausrajapinta on vaihtoehto.
curl vko 5	<i>Komentorivin osoitehakija.</i> Komento, joka hakee osoitteen ilman selainta ja tulostaa vastauksen. Sillä testataan rajapintaa suoraan: curl "http://localhost:8000/api/daylight?city=Helsinki&year=2026".
ominaisuushaara vko 6	<i>Feature branch.</i> Oma Git-haara yhdelle ominaisuudelle. Työ tehdään haarassa ja yhdistetään main-haaraan pull requestilla, kun se on valmis.
pull request vko 6	<i>Yhdistämispyyntö.</i> GitHubin pyyntö yhdistää haara pääversioon. Siinä näkyvät muutokset ja kuvaus. Katselmoi sen itse ennen yhdistämistä. Työviikolla 11 kahden pull requestin yhdistäminen tuottaa merge-konfliktin, joka ratkaistaan käsin.
hook vko 7	<i>React-hook.</i> Funktio, jolla komponentti käyttää tilaa (useState) tai sivuvaikutuksia (useEffect). Oma hook useLocations kokoaa paikkojen tilan yhteen paikkaan.
tooltip vko 8	<i>Vihjeruutu.</i> Ruutu, joka avautuu hiiren tai kosketuksen alle kaaviossa ja näyttää päivän, paikan ja keston, esimerkiksi "21. kesäkuuta · Rovaniemi · 24 h".
Vitest vko 8	<i>TypeScript-testiajuri.</i> Viten kanssa toimiva testityökalu, joka ajaa format.ts- ja intersections.ts-moduulien yksikkötestit komennolla npm test.
leikkauspiste vko 9	<i>Kahden paikan yhtä pitkä päivä.</i> Päivä, jona kahden paikan päivänvalo on yhtä pitkä tai lähes yhtä pitkä (erotus alle toleranssin). Peräkkäiset osumat yhdistetään yhdeksi merkiksi.
toleranssi vko 9	<i>Sallittu erotus.</i> Kuinka monta minuuttia kahden paikan päivänvalo saa erota, jotta päivä lasketaan leikkauspisteeksi. Oma päätös, joka perustellaan ja testataan.
merge-konflikti vko 11	<i>Yhdistämisen ristiriita.</i> Syntyy, kun kaksi haaraa on muuttanut samaa kohtaa tiedostossa. Git pysähtyy ja pyytää valitsemaan, mikä versio jää. Ratkaisu tehdään käsin ja committataan.
Lighthouse vko 12	<i>Selaimen laatumittari.</i> Chromen kehittäjätyökalujen raportti, joka mittaa saavutettavuuden, suorituskyvyn ja parhaat käytännöt. Ajetaan ennen ja jälkeen korjausten.
XSS vko 13	<i>Cross-site scripting.</i> Hyökkäys, jossa käyttäjän syöte (esimerkiksi paikan nimi) suoritetaan koodina sivulla. Testataan syöttämällä script-tagin nimeksi ja tarkistamalla, että se näkyy tekstinä.
CORS vko 13	<i>Selaimen alkuoperaatio.</i> Selain estää pyynnöt toiseen osoitteeseen, ellei palvelin salli niitä. Yhden palvelimen mallissa asetusta ei tarvita; kahden palvelimen mallissa PHP:n pitää sallia frontendin osoite.
regressiotesti vko 14	<i>Vanha testi uudelleen.</i> Korjauksen jälkeen aiemmat testit ajetaan uudelleen, jotta korjaus ei riko aiemmin toiminutta. Viimeinen askel jokaisessa virheenkorjausketjussa.
refaktorointi vko 15	<i>Rakenteen parannus ilman toiminnan muutosta.</i> Koodi järjestetään uudelleen niin, että se toimii samalla tavalla mutta on helpompi lukea ja ylläpitää. Testit todistavat, että toiminta ei muuttunut.
RC vko 16	<i>Release candidate, julkaisuehdokas.</i> Lähes valmis versio, jota ulkopuolinen testaa ennen lopullista julkaisua. v1.0-rc1 on ensimmäinen julkaisuehdokas.
Git-tag vko 16	<i>Nimetty versio historiassa.</i> Merkki, joka kiinnittää tietyn commitin versionumeroon, esimerkiksi v1.0-rc1 tai v1.0. Tagista tehdään GitHubissa release.
savutesti vko 17	<i>Nopea tarkistus julkaisun jälkeen.</i> Muutama tärkein testipolku ajetaan julkaistulle versiolle heti julkaisun jälkeen: paikan lisäys, tooltip, vuosivalinta.

Näyttömatriisi – 32 osaamisvaatimusta

Rasti vasta, kun vaatimukselle on täsmällinen työnäyte: linkki, commit, kuva, testirivi tai muistio. Sama työnäyte voi kelvata useaan kohtaan.

Ohjelmointi

- **käyttää ohjelmointieditoria tai kehitysympäristöä** – VS Code, Viten kehityspalvelin, PHP:n sisäänrakennettu palvelin ja selaimen kehittäjätyökalut käytössä työviikolta 1: versiotaulukko, kuvakaappaukset ja käynnistyskomennot README:ssä.
- **etsii ja korjaa virheitä ohjelmakoodista** – Kolme täydellistä virheenkorjausketjua (työviikot 14 ja 17; havainnot myös 4–5, 8–9 ja 16): havainto, toisto, syy, korjauscommit, uusintatesti ja regressiotesti.
- **testaa ohjelman toimintoja** – Testiraportti työviikolta 14: 16 testitapausta odotusarvoineen ennen ajoa (T01–T16); laskenta, rajapinta, muotoilu, leikkauspisteet, karkausvuosi ja virhetilanteet testattuina; ajot työviikoilla 4–9, 11 ja 13.
- **käyttää rakenteista ohjelmointia toteutuksissa** – Päivänvalolaskenta omana PHP-moduulina työviikolla 4 (funktiot, ehdot, rajatarkistus, karkausvuosi), muotoilu- ja leikkauspistemoduulit TypeScriptillä työviikoilla 8 ja 9; moduulijako client, server ja project-docs.
- **kirjoittaa ylläpidettävää ohjelmakoodia** – Kolme refaktorointia työviikolla 15 ohjaajan koodikatselmoinnin nimetyistä havainnoista, ennen ja jälkeen -diffeineen ja perusteluineen.
- **tulkitsee suunnitelmia ja toteuttaa käyttöliittymän tai sen osia** – Lomake, paikkalista ja kaavionäkymä toteutettu työviikolla 7 työviikon 2 rautalankojen mukaan Tailwindilla ilman valmista komponenttikirjastoa; mobiili ja saavutettavuus työviikolla 12.
- **tulkitsee suunnitelmia ja toteuttaa ohjelmiston toimintoja** – Paikan lisäys, poisto ja duplikaatin esto työviikolla 7 sekä vuosivalinta työviikolla 11 käyttäjätarinoiden ja hyväksymiskriteerien perusteella; rajapinnan toiminnot työviikolla 5. Issue → commit -ketju näkyvissä.
- **sopii tehtävistä tiimin muiden jäsenten kanssa** – Viikkopalaveri ohjaajan kanssa työviikoilta 2–17: sovitut tehtävät, arviot ja edellisen viikon toteuma kirjattuina issueihin, ohjaajan nimi ja päivä päiväkirjassa.
- **etsii ratkaisuvaihtoehtoja ja ratkoo ongelmia yhdessä tiimin kanssa** – Kaksi vertailua työviikolla 2 (laskentatapa; paikkakuntatiedon muoto) ja yhteinen päätös ohjaajan kanssa kirjattuna; yhdessä ratkotut ongelmat nimettyinä työviikoilla 3 (julkaisu), 4 (napapiirin laskenta-ansat) ja 11 (merge-konflikti).
- **arvioi ratkaisujen toimivuuden yhdessä tiimin kanssa** – Katselmointi työviikolla 10: ymmärtääkö asiakkaan roolissa oleva henkilö kaavion ilman selityksiä ja mitä muutetaan. Muistio ja priorisoitu muutoslista, todennus työviikolla 11.
- **arvioi omaa toimintaa tiimin jäsenenä** – Itsearviointi työviikolla 18 kolmesta nimetystä tilanteesta (työviikon 10 palaute, työviikon 16 testajan epäröinti, yksi viikkopalaveri, jossa sovittu muuttui) ja ohjaajan vastakommentti.

Ohjelmistokehittäjänä toimiminen

- **selvittää kehitystiimin kanssa asiakkaan tarpeet** – Toimeksianto purettu käyttäjätarinoiksi ja käyttäjäryhmiksi työviikolla 2; kysymyslista ja kirjatut vastaukset, rajausta sovittu ja täydennetty katselmoinnissa työviikolla 10.
- **viestii tekniset asiat asiakaslähtöisesti** – Katselmointiesittely työviikolla 10 ja luovutusviesti ohjaajalle asiakkaan roolissa työviikolla 17 ilman teknistä sanastoa, sekä vastaus asiakkaan kysymykseen.
- **osallistuu version katselmointiin** – Kaksi katselmointia: väliversion asiakaskatselmointi työviikolla 10 ja julkaisuehdokkaan julkaisutestaus työviikolla 16. Palaute ja sovitut muutokset kirjattuina.
- **asettaa kehitystiimin kanssa toteutettavat toiminnot tärkeysjärjestykseen** – P0-, P1- ja P2-priorisointi ohjaajan kanssa työviikolla 2; P0-ydin eli oikea päivänvalodata, usea paikka ja virheilmoitus toteutettu ensin.
- **jakaa kehitystiimin kanssa toteutettavat toiminnot tehtäviksi** – Käyttäjätarinat pilkottu issueiksi työviikolla 2 hyväksymiskriteereineen, noin puolen tai yhden päivän kokoisina; issue-taulu koko projektin ajan.
- **suunnittelee ja arvioi kehitystiimin kanssa tehtävien toteuttamista** – Työmääräarviot ja toteumat issueissa jokaisen viikkopalaverin jälkeen työviikolta 2; arvio vs. toteuma -vertailu työviikolla 7 ja suunnitelman päivitys, kun arvio petti.
- **kehittää ohjelmiston toimintalogiikkaa** – Päivänvalolaskenta rajoin 0–1440 ja karkausvuosin työviikolla 4, haun normalisointi ja syötteiden validointi työviikolla 5, leikkauspisteiden laskenta toleranssilla työviikolla 9, yhden paikan virheen käsittely työviikolla 13.
- **valitsee ohjelmistoon sopivan tietovaraston** – Paikkakuntatiedon muodon vertailu työviikolla 2 (JSON-tiedosto vai SQLite; ulkoinen geokoodausrajapinta vain perustellusti) haun, kattavuuden, lisenssin ja verkkoriippuvuuden perusteella; perusteltu valinta suunnitelmassa.

- **toteuttaa yhteyden tietovarastoon** – Paikkakuntatiedon luku ja haku nimellä työviikolla 5 (normalisointi, puuttuva paikka) omassa Geocoder-moduulissa; SQLite-polulla yhteyskerros, JSON-polulla latausmoduuli ja tiedoston rakenne; tietolähteen lisenssi kirjattuna.
- **hyödyntää rajapintoja ja käsittelee tietoa** – Oman rajapinnan kutsu fetchillä työviikoilla 3 ja 6, JSON-vastauksen muunnos kaavion sarjaksi tyytittynä, lataus- ja virhetilat käsiteltynä; uudelleenhaku vuosivalinnalla työviikolla 11; ulkoinen geokoodausrajapinta työviikolla 5, jos käytössä.
- **arvioi ohjelmiston tietoturvaa** – Tietoturva-arvio työviikolla 13 oman sovelluksen uhkalistana (uhka, testi, tulos, toimenpide): syötteiden rajat, XSS-testi paikan nimellä, ulkoisen palvelun aikakatkaisu, salaisuudet poissa repositorysta; ajettu syötetaulukko T15.
- **käyttää versionhallintaa** – Git koko projektin ajan työviikolta 1: tarkoituksenmukaiset commitit, etärepository, ominaisuushaarat ja pull requestit työviikolta 6, tagit. Historia työnäytteenä, koonti työviikoilla 11 ja 15.
- **liittää ohjelman osan olemassa olevaan versioon** – Palautemuutos ominaisuushaarassa työviikolla 11: pull request, itsekselmointi, konfliktin ratkaisu ja merge pääversioon.
- **julkaisee ohjelman tuotantoympäristöön** – Tuotantobuild ja julkaisu valittuun alustaan: ensijulkaisu työviikolla 3, julkaisuehdokas työviikolla 16 ja v1.0 työviikolla 17 julkisessa osoitteessa.

Ohjelmiston toteuttaminen ohjelmistokomponenttikirjastolla

- **ottaa käyttöön ja konfiguroi ohjelmistokomponenttikirjaston käyttöön soveltuvan kehittämisympäristön** – React + TypeScript + Vite -projektin luonti työviikolla 1; Viten proxy PHP-rajapintaan ja tuotantobuildin asetukset työviikolla 3; Tailwindin konfigurointi työviikolla 7.
- **selvittää ohjelmistokomponenttikirjaston tarjoamat mahdollisuudet ja rajoitteet** – Selvitys taulukkona omista P0-tarinoista työviikolla 3: mikä osa tehdään Reactilla ja Vitellä, mikä tulee ulkopuolelta ja mistä; kaaviokirjaston näppäimistörajoite ja kiertoreitti kirjattuina työviikoilla 8 ja 12.
- **käyttää ohjelmistokomponenttikirjaston tärkeimpiä toimintoja ja työkaluja** – Komponentit, propsit, tila ja hookit (useState, useEffect, oma useLocations), ehdollinen renderöinti ja lomakkeen käsittely osoitettuna työviikoilla 3, 6–8 ja 11–12.
- **tuo kehittämisympäristöön ulkoisia komponentteja** – Kaksi perusteltua ulkoista komponenttia: kaaviokirjasto työviikolla 6 (vertailu kokeilusarakkeella, koko, lisenssi, riippuvuudet) ja Tailwind CSS työviikolla 7 (mitä se ratkaisee lomakkeessa ja listassa, konfigurointi).
- **suunnittelee, toteuttaa ja testaa ohjelmiston ohjelmistokomponenttikirjastoa käyttäen** – Komponenttijako ja vastuut suunnitelmassa työviikolla 2, sovellus toteutettu Reactilla käyttäjätarinoiden mukaan työviikoilla 6–9 ja 11–12, omat ja kirjastoon liittyvät ratkaisut testattuina työviikolla 14.
- **julkaisee ohjelmiston asiakkaan ympäristöön** – Viten tuotantobuild ja PHP-rajapinta julkaistuna sovittuun alustaan: julkaisuehdokas tagilla v1.0-rc1 työviikolla 16 ja v1.0 työviikolla 17.
- **dokumentoi ohjelmiston sovitulla tavalla** – README työviikolla 15: käyttöönotto tyhjään ympäristöön, kehityskomennot, rajapinnan kuvaus esimerkkivastauksella ja riippuvuustaulukko lisensseineen; lisäksi asiakkaan pikaohje.

Muista: sivuston rastit ja kentät tallentuvat vain selaimeen. Ne eivät siirry opettajalle eivätkä korvaa Gitissä olevaa työtä.